C. COMPUTAÇÃO
UNIPAC

Computação Gráfica

Textura

Professor:

Dr. Rodrigo Fenandes dos Santos

CENTRO UNIVERSITÁRIO UNIPAC
SCIENTIA LIBERTAS ET FIDES
PRESIDENTE ANTÔNIO CARLOS

unipac.br
Barbacena

Da imagem ao retângulo e ao cubo em rotação

unipac.br
Barbacena

C. COMPUTAÇÃO
UNIPAC

COMPUTAÇÃO GRÁFICA

Objetivos da aula

Ao final, você será capaz de:

01 **Compreender texturas**
Identificar textura, texel e coordenadas UV.

02 **Relacionar os componentes**
Conectar Texture e TextureMaterial à geometria e aos shaders.

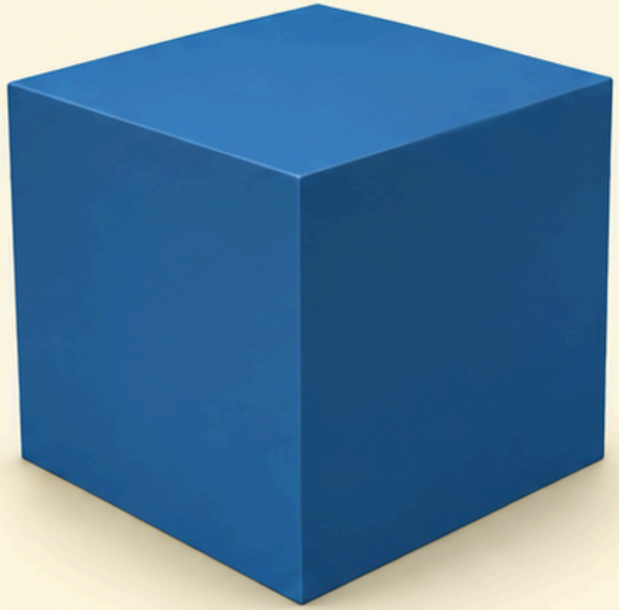
03 **Executar exemplos em Python**
Exibir um retângulo texturizado e um cubo em rotação.

04 **Aplicar na prática**
Adaptar o cubo em dupla nos 20 minutos finais.

COMPUTAÇÃO GRÁFICA

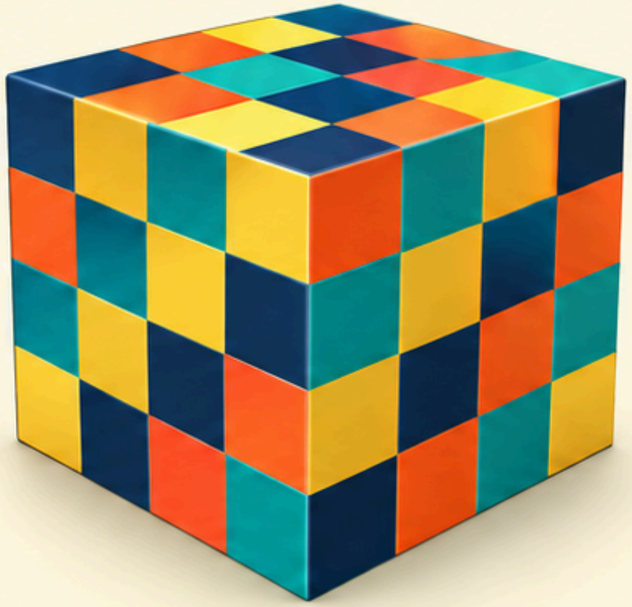
Cor uniforme e textura

A mesma geometria pode receber aparências diferentes.



Cor uniforme

Cobre a superfície com um único valor de cor.



Textura

Permite variar a cor conforme a posição na imagem.

A geometria é a mesma. O que muda é a aparência da superfície.

COMPUTAÇÃO GRÁFICA

O que é uma textura?

01

Dados para o shader

Uma textura armazena dados que o shader consulta durante a renderização.

02

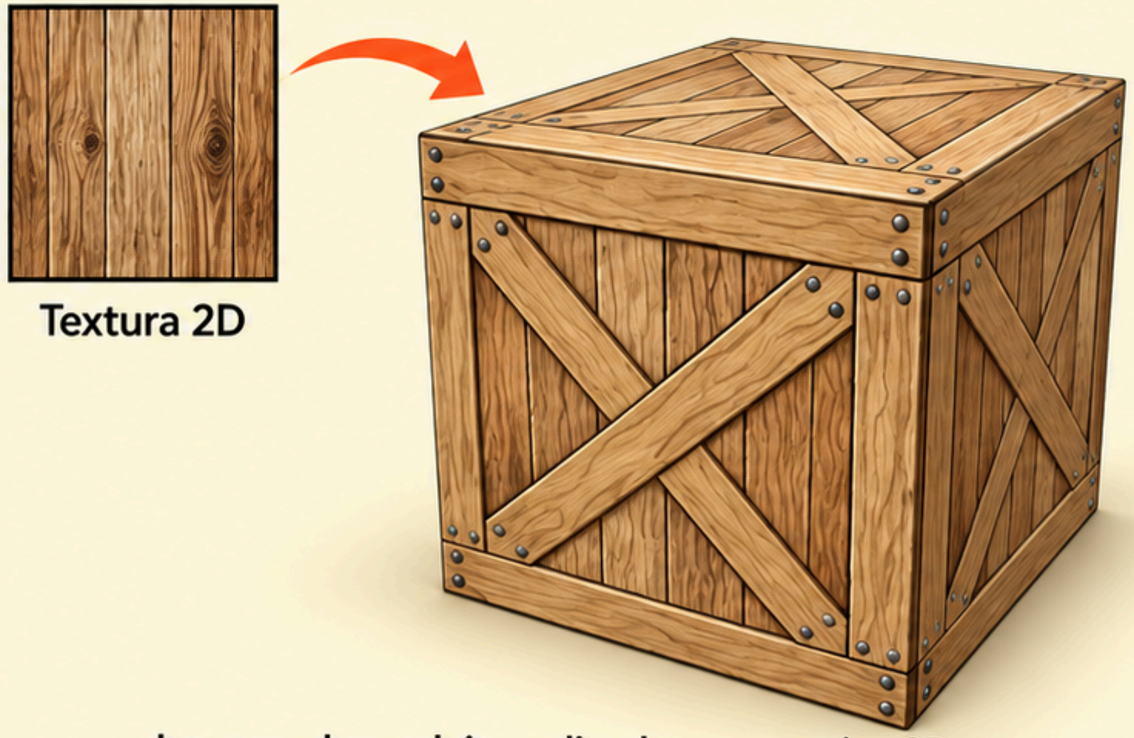
Uma imagem 2D

Nesta aula, esses dados vêm de uma imagem 2D.

03

Aplicações comuns

Padrões de madeira, pisos, rótulos e interfaces.



Textura 2D

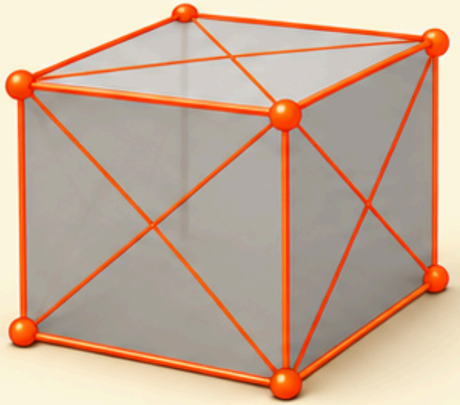
Imagem de madeira aplicada a uma caixa 3D

A textura define detalhes da aparência da superfície.

COMPUTAÇÃO GRÁFICA

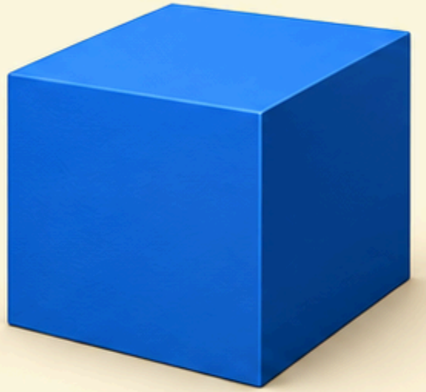
Geometria, cor e textura

Geometria



Define os vértices e os triângulos da superfície.

Cor uniforme



Fornece uma mesma cor de base para o material.

Textura

Fornece dados de cor consultados com coordenadas UV.

Mesh reúne geometria e material

Mesh

Geometry
Posições + UV

+

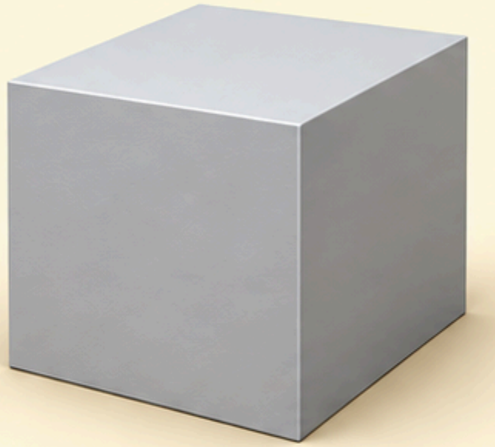
Material
Shaders + uniforms

No exemplo, Mesh reúne a geometria e o material texturizado.

COMPUTAÇÃO GRÁFICA

Analogia: revestindo uma caixa

01 Caixa → Geometria



A caixa representa a geometria.

02 Folha → Textura

A folha impressa representa a imagem da textura.

03 Encaixe → UV

As marcações de encaixe representam as coordenadas UV.

E o material?

O material orienta como usar a imagem sobre a superfície.

A caixa mantém a forma; o revestimento muda a aparência.

COMPUTAÇÃO GRÁFICA

Pixels e texels

01 Pixel

Elemento de uma imagem, inclusive do resultado na tela.

Imagem na tela

02 Texel

Elemento da imagem quando ela funciona como textura.

Imagem da textura

Como a GPU usa a textura

A GPU amostra texels para calcular a cor dos fragmentos.

Ampliação

Filtragem

Um texel não corresponde sempre a um único pixel da tela.

COMPUTAÇÃO GRÁFICA

PERGUNTA 1

O papel da textura

Ao aplicar uma imagem de madeira a um cubo, o que muda neste exemplo?

A. A quantidade de vértices do cubo.

B. A cor calculada ao longo da superfície.

C. A posição da câmera automaticamente.

Escolha uma alternativa e explique seu raciocínio.

COMPUTAÇÃO GRÁFICA

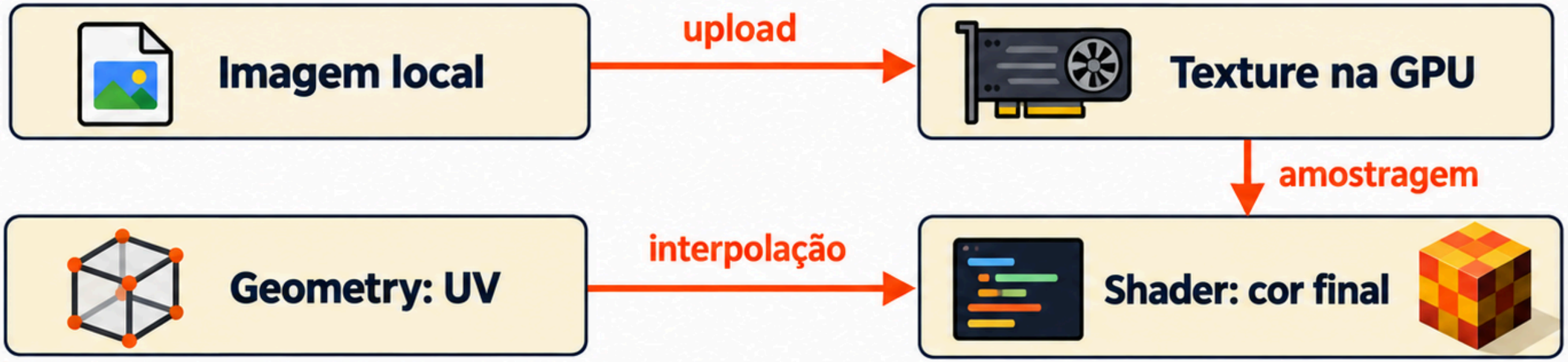
Percurso da imagem na renderização

01 Texture
carrega a imagem e envia seus dados para a GPU.

02 Geometry
fornece posições e coordenadas UV.

03 TextureMaterial
associa os shaders à textura.

04 Renderer
desenha a Mesh com a câmera da cena.



COMPUTAÇÃO GRÁFICA

Responsabilidade da classe Texture

01 **surface**
Guarda os dados da imagem no Pygame.

02 **textureRef**
Identifica o objeto de textura no OpenGL.

03 **loadImage()**
Lê o arquivo local.

04 **uploadData()**
Envia pixels e configura filtros e repetição.

O construtor carrega a imagem e prepara a textura para uso.

core/texture.py

```
grid = Texture(caminho)
# loadImage() e uploadData()
# acontecem no construtor.
```

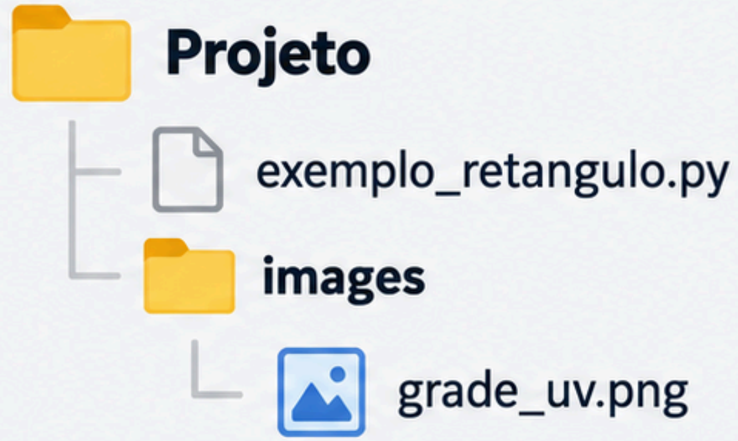
COMPUTAÇÃO GRÁFICA

Carregamento de uma imagem local

01 O caminho parte da localização do programa.

02 A imagem precisa acompanhar o projeto.

03 Pygame lê o arquivo e cria uma Surface em memória.



exemplo_retangulo.py

```
pasta = Path(__file__).resolve().parent
grid = Texture(
    pasta / 'images' / 'grade_uv.png'
)
```

O caminho é construído a partir da pasta do script.

COMPUTAÇÃO GRÁFICA

Envio da imagem para a GPU

01 RGBA guarda os componentes vermelho, verde, azul e alpha.

02 A conversão inverte as linhas para alinhar a imagem ao sistema UV.

03 glTexImage2D informa dimensões, formato e dados dos texels.

core/texture.py

```
pixelData = pygame.image.tostring(
    self.surface, 'RGBA', True
)
glBindTexture(GL_TEXTURE_2D,
              self.textureRef)
```

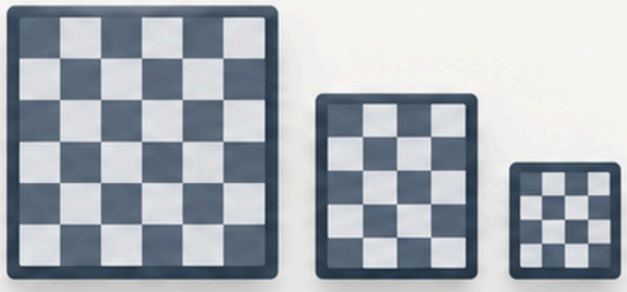
True inverte a imagem verticalmente.

Upload na classe Texture

01 A textura vinculada recebe os dados da imagem.

02 GL_UNSIGNED_BYTE usa um byte por componente de cor.

03 Os mipmaps fornecem versões menores para a minificação.



core/texture.py

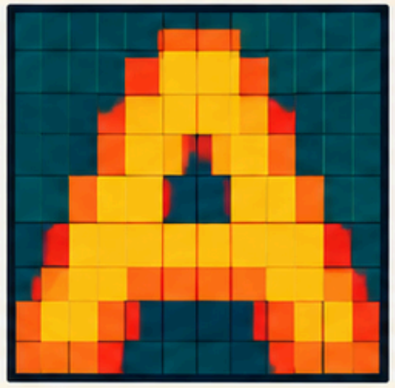
```
glTexImage2D(
    GL_TEXTURE_2D, 0, GL_RGBA,
    width, height, 0, GL_RGBA,
    GL_UNSIGNED_BYTE, pixelData
)
glGenerateMipmap(GL_TEXTURE_2D)
```

COMPUTAÇÃO GRÁFICA

Filtros e repetição da textura

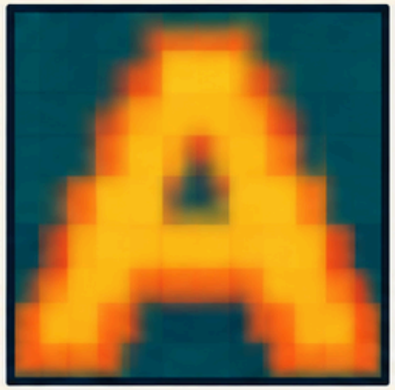
01 GL_NEAREST

Destaca os texels ao ampliar.



02 GL_LINEAR

Interpola cores de texels próximos.

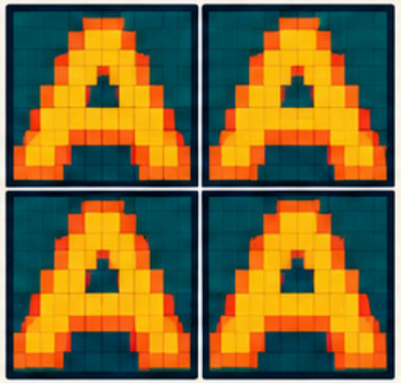


03 GL_LINEAR_MIPMAP_LINEAR

Combina níveis de mipmap ao reduzir.

04 GL_REPEAT

Repete a imagem para UV fora de [0,1].



Filtros controlam a amostragem; repetição controla o uso das coordenadas UV.

COMPUTAÇÃO GRÁFICA

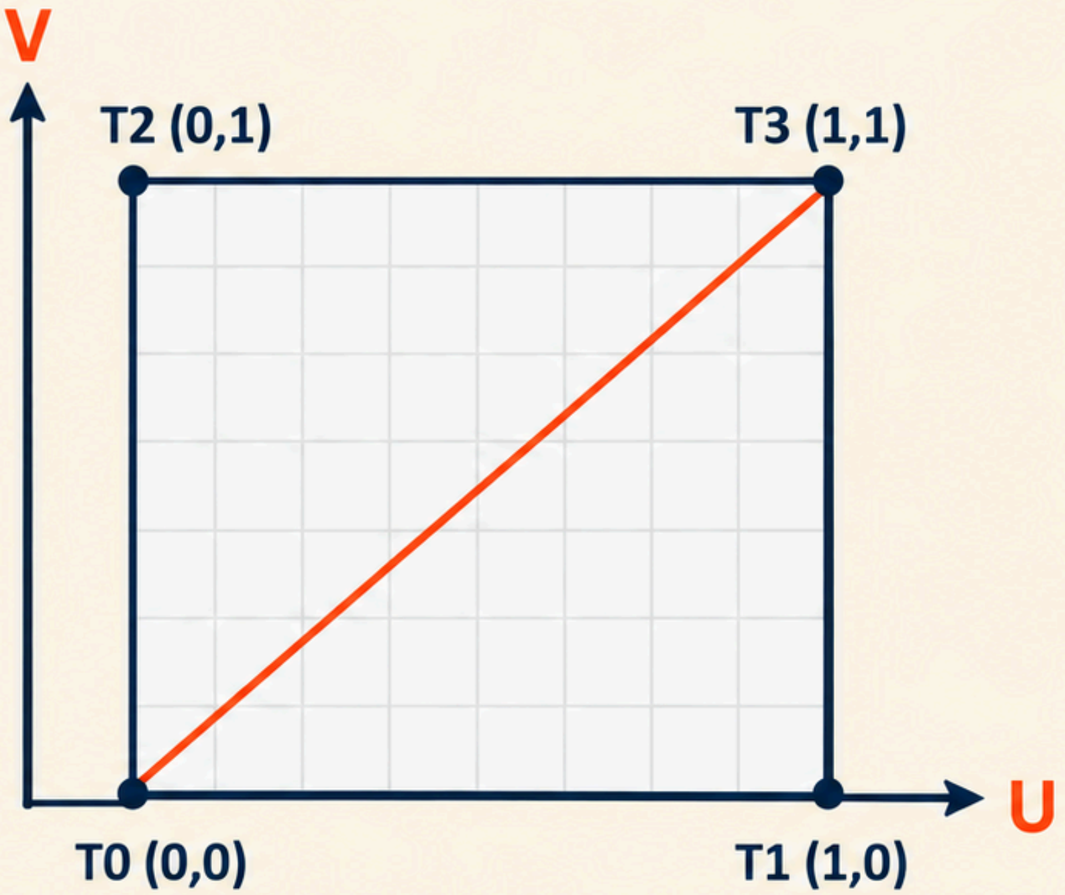
Coordenadas UV

01 UV indica onde consultar a imagem da textura.

02 U representa a direção horizontal.
V representa a vertical.

03 O intervalo de 0 a 1 percorre uma imagem completa.

04 Cada vértice recebe uma coordenada UV associada à sua posição.



COMPUTAÇÃO GRÁFICA

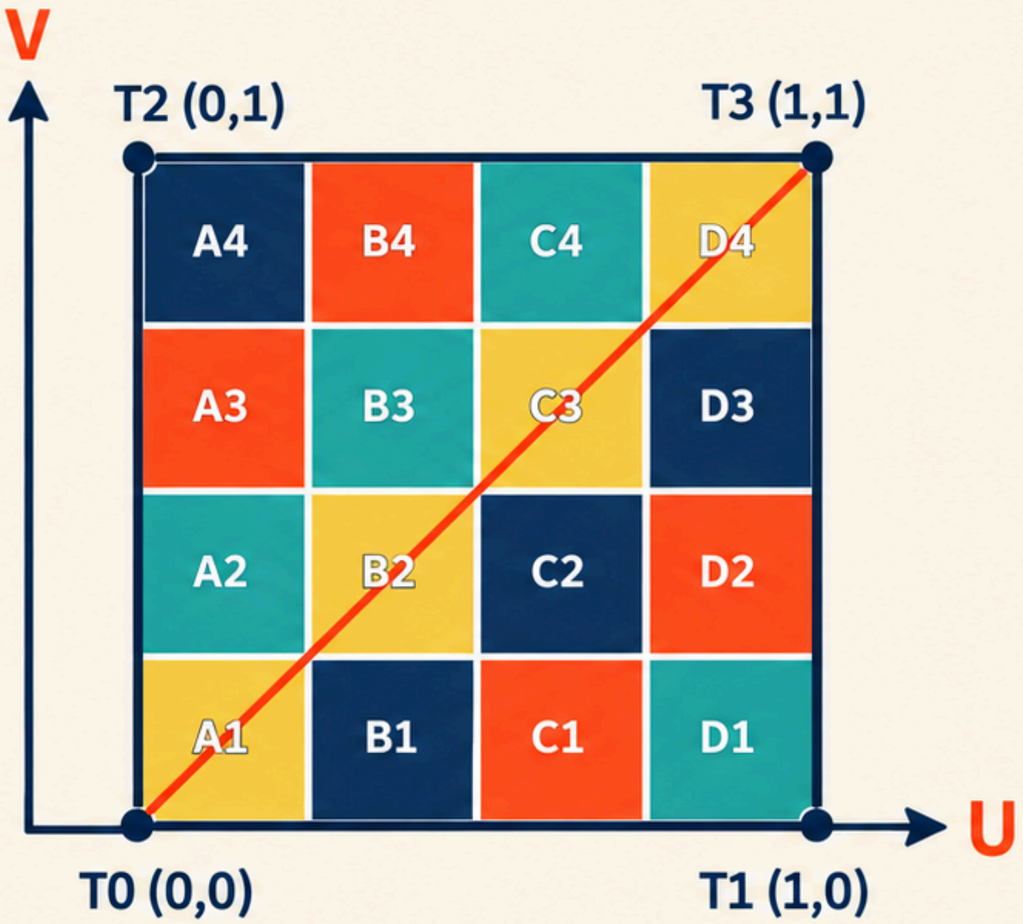
Pontos de referência na imagem

(0,0): canto inferior esquerdo.

(1,0): canto inferior direito.

(0,1): canto superior esquerdo.

(1,1): canto superior direito.



COMPUTAÇÃO GRÁFICA

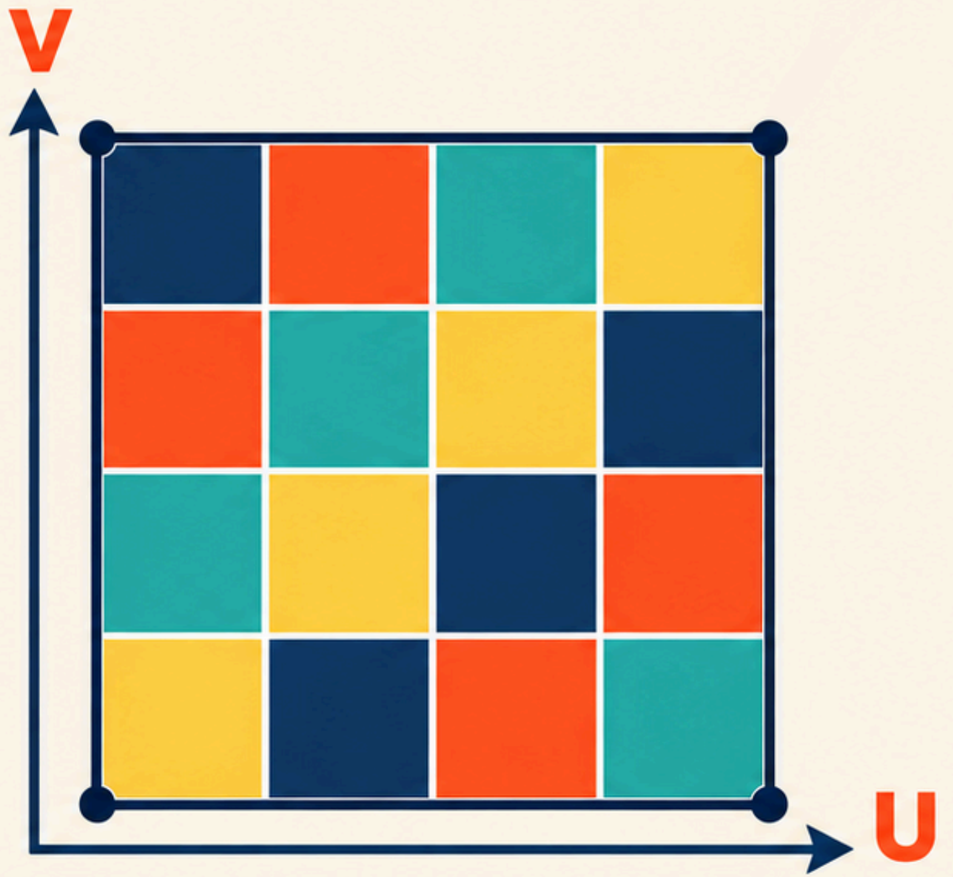
Pergunta 2: interpretação de UV

Qual coordenada aponta para o centro da imagem?

A) (0,0)

B) (1,1)

C) (0.5,0.5)



COMPUTAÇÃO GRÁFICA

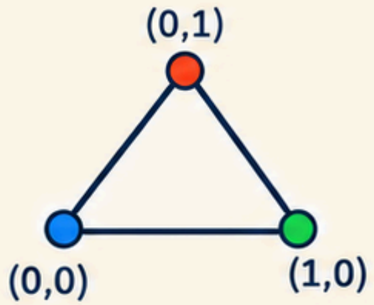
Papel dos shaders

O **vertex shader** calcula a posição do vértice na projeção.

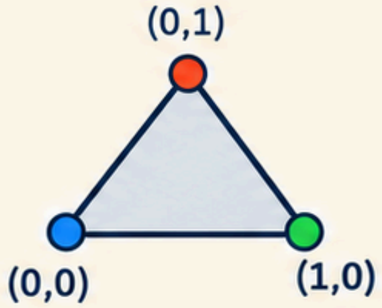
Ele também encaminha as **coordenadas UV** para a rasterização.

O **fragment shader** consulta a textura e produz uma cor.

vertexUV
UV de cada vértice

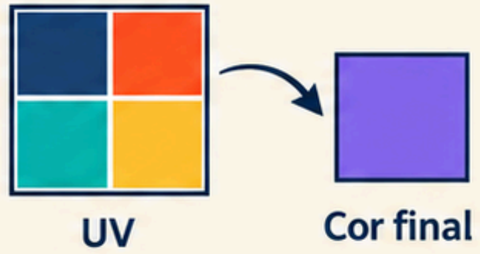


Vertex shader
Encaminha as UV



Rasterização
Interpola as UV

Fragment shader
Consulta a textura e produz a cor



COMPUTAÇÃO GRÁFICA

Vertex shader e passagem de UV

vertexUV entra como atributo vec2.

UV sai do vertex shader e chega interpolado ao fragment shader.

shaders/texture.vert

```
in vec2 vertexUV;
out vec2 UV;
// Dentro de main():
UV = vertexUV * repeatUV + offsetUV;
```

repeatUV: controla a repetição.

offsetUV: desloca a textura.

COMPUTAÇÃO GRÁFICA

Fragment shader e amostragem

sampler2D indica qual unidade de textura consultar.

texture() amostra a imagem usando o UV do fragmento.

fragColor recebe a cor resultante.

shaders/texture.frag

```
uniform sampler2D textureSampler;
in vec2 UV;
out vec4 fragColor;
// Dentro de main():
vec4 color = texture(textureSampler, UV);
fragColor = color;
```

COMPUTAÇÃO GRÁFICA

Integração entre geometria e material

Geometry fornece vertexPosition e vertexUV.

TextureMaterial guarda os shaders e seus uniforms.

Mesh associa os atributos ao programa do material.

Uniform liga textureRef à unidade 1 e envia o número 1 ao sampler.

unipac.br
Barbacena

unipac.br
Barbacena

unipac.br
Barbacena

unipac.br
Barbacena

unipac.br
Barbacena

unipac.br
Barbacena

CONTATO

rodrigofernandes@unipac.br

OBRIGADO!!